

LINUX NETWORKING COMMANDS

Comprehensive Lab Practical & Output Interpretation Manual

1. Introduction & Lab System Setup

This manual provides a hands-on technical breakdown of 15 essential Linux networking utilities used for administration, troubleshooting, performance benchmarking, and security auditing. Commands that modify network interfaces or capture low-level raw socket packets require elevated privileges via `sudo`.

2. Interface Configuration & Status

A. command: `ip` (The Modern Standard)

The core replacement for the legacy net-tools suite. Used to configure, inspect, and route traffic across system interfaces.

```
$ ip a
```

Expected Terminal Output:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:54:00:12:34:56 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.42/24 brd 192.168.122.255 scope global dynamic eth0
        valid_lft 3582sec preferred_lft 3582sec
```

Output Field Interpretation:

- **<UP, LOWER_UP>** : **UP** indicates the administrative state is active. **LOWER_UP** confirms physical layer continuity (cable plugged/signal detected).
- **mtu 1500** : Maximum Transmission Unit. The maximum payload size in bytes per packet frame.
- **inet 192.168.122.42/24** : The assigned IPv4 address. The CIDR notation **/24** defines a subnet mask of **255.255.255.0**.

B. command: `ifconfig` (The Legacy Standard)

Deprecated but heavily encountered in legacy infrastructures to inspect interface statistics.

```
$ ifconfig eth0
```

Expected Terminal Output:

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
      inet 192.168.122.42 netmask 255.255.255.0 broadcast 192.168.122.255
      ether 52:54:00:12:34:56 txqueuelen 1000 (Ethernet)
      RX packets 10432 bytes 12453021 (12.4 MB)
      TX packets 8421 bytes 943021 (943.0 KB)
```

Output Field Interpretation:

- **RX packets / TX packets** : Counters tracking successfully Received (RX) and Transmitted (TX) traffic packets. Increments confirm live operational flow.

C. command: **nmcli** (NetworkManager CLI)

Manages structural connection profiles managed by the system NetworkManager daemon.

```
$ nmcli dev status
```

DEVICE	TYPE	STATE	CONNECTION
eth0	ethernet	connected	Wired conn 1
lo	loopback	unmanaged	--

D. command: **ifplugstatus** (Link State Detection)

Queries hardware register parameters directly to report if an active link beat exists on a physical interface port.

```
$ ifplugstatus eth0
```

```
eth0: link beat detected
```

3. Connectivity & Path Diagnostics

A. command: **ping**

Utilizes ICMP Echo Requests to map layer-3 diagnostic reachability to a remote host node.

```
$ ping -c 3 google.com
```

```
PING google.com (142.250.190.46) 56(84) bytes of data.
64 bytes from ord38s31-in-f14.1e100.net (142.250.190.46): icmp_seq=1 ttl=116 time=12.4 ms
64 bytes from ord38s31-in-f14.1e100.net (142.250.190.46): icmp_seq=2 ttl=116 time=11.8 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 11.821/12.108/12.412/0.241 ms
```

Output Field Interpretation:

- **ttl=116** : Time-to-Live metric. Packets decrement by 1 at every routing hop to mitigate loops. This response indicates the target is ~12 network hops away.
- **time=12.4 ms** : Total Round-Trip Time (RTT) for execution.
- **0% packet loss** : Indicates a completely clear, non-congested route path.

B. command: **tracert**

Maps network paths sequentially by incrementing the TTL value field from 1 onwards, exposing intermediate routers.

```
$ tracert -n google.com
```

```
tracert to google.com (142.250.190.46), 30 hops max
 1  192.168.122.1  0.312 ms  0.288 ms
 2  10.0.0.1  1.421 ms  1.399 ms
 3  142.250.190.46  12.102 ms  11.984 ms
```

C. command: **tr** (My Traceroute)

Combines real-time active traceroute polling alongside responsive ping metrics to expose upstream node packet drop.

```
$ tr --report -c 5 google.com
```

HOST: localhost	Loss%	Snt	Last	Avg	Best	Worst
1. -- 192.168.122.1	0.0%	5	0.3	0.3	0.2	0.4
2. -- 10.0.0.1	0.0%	5	1.4	1.5	1.4	1.7
3. -- 142.250.190.46	0.0%	5	12.1	12.0	11.8	12.3

4. Connections & Live Bandwidth Tracking

A. command: **ss**

Displays ongoing infrastructure sockets, route metrics, and socket states.

```
$ sudo ss -tulpn
```

```
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/
Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:* LISTEN      842/sshd
tcp6       0      0 :::80                   :::* LISTEN      911/apache2
```

Output Field Interpretation:

- **0.0.0.0:22** : Indicates port 22 (SSH) is binding and actively listening on all local loopback and interface network configurations.
- **LISTEN** : Network sockets configured to accept incoming inbound handshakes.
- **842/sshd** : The target process ID and daemon holding the socket configuration open.

B. command: **iftop** & **bmon**

iftop isolates bandwidth consumption metrics linearly by remote host connections, while **bmon** builds raw throughput visualization charts directly in-terminal.

```
$ sudo iftop -i eth0 -N -n -B
```

```
192.168.122.42          => 93.184.216.34          120KB   98KB   84KB
```

5. Domain Resolution & Security Profiling

A. command: **nslookup** & **whois**

nslookup isolates target query maps on DNS servers to map hostnames to IP pointers. **whois** provides operational owner lookup info from domain registries.

```
$ nslookup github.com
```

```
Non-authoritative answer:
Name:   github.com
Address: 140.82.112.4
```

B. command: **nmap**

Scans ports and target network spaces to discover active services.

```
$ nmap 192.168.122.1
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
```

C. command: `tcpdump`

Intercepts, logs, and parses network frame sequences moving across interfaces via low-level network hooks.

```
$ sudo tcpdump -i eth0 -c 2 icmp
```

```
22:58:10.102431 IP 192.168.122.42 > 142.250.190.46: ICMP echo request, id 412, seq 1, length 64
22:58:10.114512 IP 142.250.190.46 > 192.168.122.42: ICMP echo reply, id 412, seq 1, length 64
```

D. command: `speedtest-cli`

Queries upstream public Speedtest infrastructure to map raw uplink/downlink WAN throughput capacity capabilities.

```
$ speedtest-cli --simple
```

```
Ping: 14.21 ms
Download: 142.55 Mbit/s
Upload: 88.12 Mbit/s
```

6. Lab Troubleshooting Practical Scenario

Scenario: Host endpoint system reports complete failure accessing a remote web resource: `example.com`. Execute this diagnostic plan sequence:

Step 1: Link Verification - Run `ifplugstatus eth0` and `nmcli dev status` to verify physical link state and administrative connection status.

Step 2: Addressing Check - Execute `ip a` to verify that a valid non-APIPA (169.254.x.x) address has been dynamically bounded via DHCP.

Step 3: Local Gateway Test - Ping the standard local route configuration gateway: `ping -c 3 192.168.122.1`.

Step 4: Resolve Names - Run `nslookup example.com` to verify name resolution is working correctly.

Step 5: Isolate Failures - Execute `mtr example.com` to precisely isolate which upstream infrastructure hop is causing the drop.